

# Ch 03: Page Tables

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Summer 2026



**EAST TEXAS A&M**

Page tables are the most popular mechanism through which an operating system provides each process with its own private address space and memory

- Page tables:
  - determine what memory addresses mean
  - determine what parts of physical memory can be accessed
  - allow **isolation** of different process' address spaces
  - allow to **multiplex** processes into a single physical memory
  - allow illusion that each process has full control of memory address space with no other processes able to access
- Page tables provide a level of indirection allowing some tricks used by xv6
  - map same memory (trampoline page) in several address spaces
  - guard kernel and user stacks with unmapped page to detect overflow
  - allocate user heap memory lazily



# Paging Hardware (1 of 4)

- RISC-V instructions (both user and kernel) manipulate virtual addresses
- The machine's RAM, or physical memory, is indexed with physical addresses
- The RISC-V page table hardware maps each virtual address to physical address
- Xv6 uses RISC-V Sv39 mode
  - only bottom 39 bits (of 64) are used
  - RISC-V page table is array of  $2^{27}$  (134, 217, 728) **page table entries** PTEs
  - Each PTE contains 44-bit physical page number PPN and flags
  - Paging hardware translates top 27 bits of virtual address into page table index to find PTE
  - Results in 56-bit physical address, 44-bits from PPN, and 12-bit offset

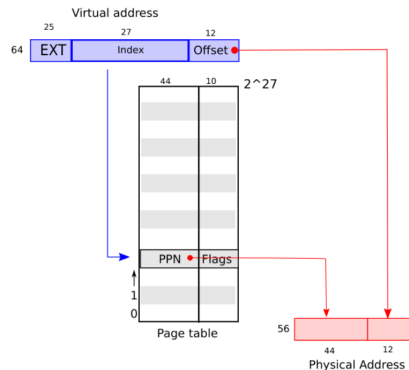


Figure 1: An abstract view of a flat page table mapping virtual to physical addresses (Cox et al., 2026, pg.32)



# Paging Hardware (2 of 4)

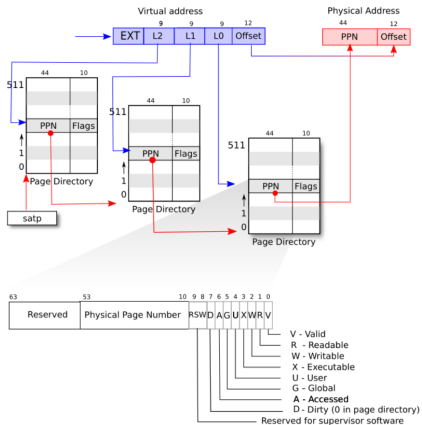


Figure 2: RISC-V address translation details (Cox et al., 2026, pg.33)

- A page table gives the operating system control over virtual-to-physical address translations
  - page size  $2^{12} = 4096$  bytes kernel/riscv.h PGSIZE
  - Such a chunk is called a **page**
- Actual page table layout is a bit more complex, three-level tree
  - root of tree is 4096-byte page table containing 512 PTEs (8 bytes or 64 bits per PTE)
  - first 9 bits of virtual address index top level page table, next mid level, etc.
    - Allows memory-efficient way of managing PTEs
  - Potentially requires 3 memory accesses for each VA → PA translation
    - RISC-V CPU uses **Translation Look-aside Buffer (TLB)**



# Paging Hardware (3 of 4)

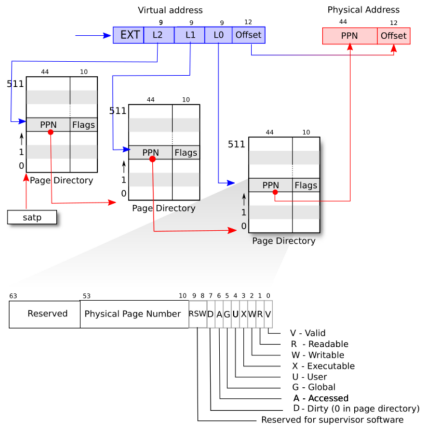


Figure 3: RISC-V address translation details (Cox et al., 2026, pg.33)

- PTE\_V Valid, indicates whether the PTE is present, if not reference causes page fault
- PTE\_R Readable, controls whether instructions are allowed to read the page
- PTE\_W Writable, controls whether instructions are allowed to write to the page
- PTE\_X Executable, controls whether CPU may interpret content of page as instructions and execute them
- PTE\_U User, controls whether instructions in user mode are allowed to access page
- PTE\_A Accessed, indicates if page has ever been accessed (read from) since created
- PTE\_D Dirty, indicates if page has been written to



# Paging Hardware (4 of 4)

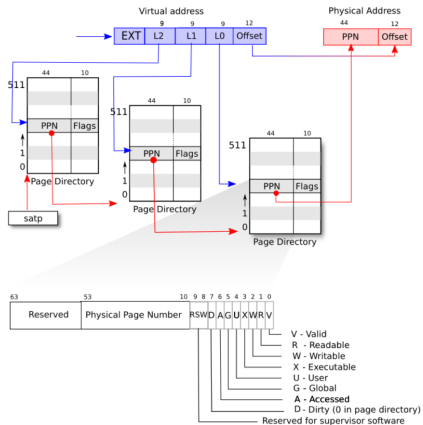
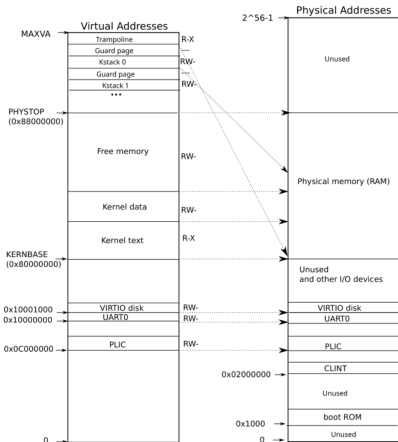


Figure 4: RISC-V address translation details (Cox et al., 2026, pg.33)

- To tell CPU to use page table, kernel writes physical address of root page-table into satp register
  - kernel/start.c disables paging initially
  - kernel/main.c calls `kvminithart()` in `kernel/vm.c` which enables kernel page table
  - Later each process has a page table created, and `satp` is set each time xv6 switches to user mode
- CPU translates all addresses by subsequent instructions using page table pointed to by `satp`
  - Each multicore CPU has own `satp` so each can run different processes with private address space
- From the kernel's point of view, page table is data stored in memory
  - kernel creates and modifies page tables using code much like any you would see for tree-shaped data structure



# Kernel Address Space

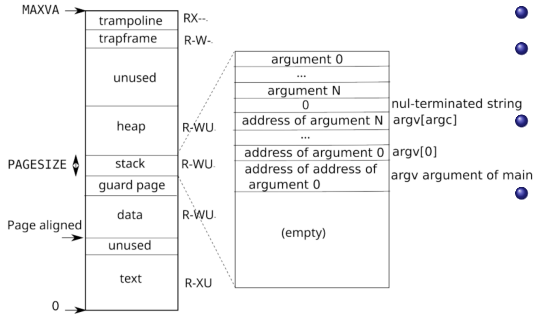


- When it starts, xv6 create a single page table describing the kernel's address space
  - configures layout of its address space to give it access to all of physical memory
  - configured to allow access to various hardware resources at predictable virtual addresses
  - configured in `kernel/memlayout.h`

Figure 5: On the left xv6's kernel virtual address space. RWX refer to PTE read, write and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.



# Process Address Space (1 of 2)



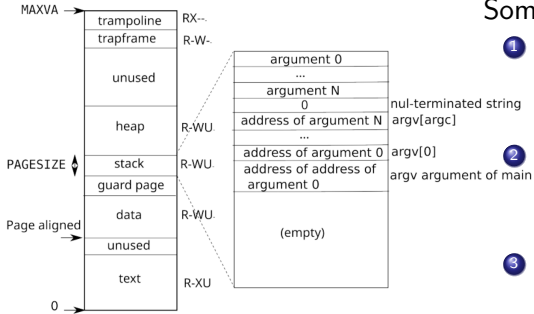
- Each process has its own page table
- When xv6 switches between processes, it also changes page tables
- User addresses start at 0 and end at MAXVA (0x4000000000) kernel/riscv.h near bottom
- A processes address space has pages with
  - text of the program (instructions, permissions are PTE\_R, PTE\_X and PTE\_U)
  - pre-initialized data (constants, global variables PTE\_R, PTE\_W, PTE\_U)
  - a page for the stack (PTE\_R, PTE\_W, PTE\_U)
  - pages for dynamic heap (PTE\_R, PTE\_W, PTE\_U)

Figure 6: A process's user address space, with its initial stack (Cox et al., 2026, pg.39)





# Process Address Space (2 of 2)



Some properties of xv6 use of page tables here:

- 1 Different process' page tables translate user addresses to different pages of physical memory, so that each process has private user memory
- 2 Each process sees its memory as having contiguous virtual addresses starting at 0, while the process's physical memory can be non-contiguous
- 3 The kernel maps a page with trampoline code at the top of the user address space (without PTE\_U), thus a single page of physical memory shows up in all address spaces, but can be used only by the kernel.

Figure 7: A process's user address space, with its initial stack (Cox et al., 2026, pg.39)



# Summary



Cox, R., Kaashoek, F., & Morris, R. (2026). *Xv6: A simple, unix-like teaching operating system*. Retrieved September 2, 2025, from <https://github.com/mit-pdos/xv6-riscv-book>

